

SQL – DML

(Data Manipulation Language)

DML, or **Data Manipulation Language**, is the part of SQL used to work with the actual data stored inside database tables.

While DDL commands such as CREATE TABLE and DROP TABLE define the structure of the database, DML commands are used to add, change, and remove the records stored in that structure.

The main DML operations are:

1. INSERT — adds new rows to a table
2. UPDATE — changes existing rows
3. TRUNCATE and DELETE — removes rows

1. INSERT

The INSERT operation is used to add new records, or rows, to a database table. The inserted values must follow the **data types** and **constraints** defined for that table.

For example, suppose we have this table:

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Major VARCHAR(50)  
);
```

To add a new student, we use INSERT:
INSERT INTO Student VALUES
 (101, 'John Doe', 'Computer Science');

If a column is designated as a **PRIMARY KEY**, it must contain a **Unique and non-NULL value**.

To make your intention clear, you can include the column names in the insert operation:

```
INSERT INTO Student (StudentID, Name, Major) VALUES  
    (101, 'John Doe', 'Computer Science');
```

You can also insert several rows at the same time:

```
INSERT INTO Student (StudentID, Name, Major) VALUES  
    (102, 'Alice Johnson', 'Computer Science'),  
    (103, 'Brian Smith', 'Biology'),  
    (104, 'Carla Davis', 'Business');
```

The values being inserted must follow the table's **data types and constraints**.

For example, if StudentID is a PRIMARY KEY, this would fail if 101 already exists:

```
INSERT INTO Student (StudentID, Name, Major) VALUES  
(101, 'Mary Smith', 'Biology');
```

*SQL is generally considered an **interpreted language** because SQL statements are processed and executed by the DBMS when they are submitted.*

However, internally, the DBMS may compile and optimize the statements into an execution plan before running them.

If for some reason, we want to change the order of the data, we can specify the order in the INSERT operation:

```
INSERT INTO Student (Name, Major, StudentID)  
VALUES ('John Doe', 'Computer Science', 1);
```

You can also omit columns when they allow NULL or have a default value.

For example:

```
INSERT INTO Student (Name, StudentID)  
VALUES ('John Doe', 1);
```

Likewise, if a column is declared NOT NULL, a required value must be supplied.

Peculiar behavior of SQLite:

Although SQL is standardized, different database management systems (DBMSs) may implement certain SQL features differently.

As a result, a statement that works in one DBMS may behave differently or require different syntax in another.

For Example:

```
INSERT INTO Student (Name, StudentID)
VALUES ('John Doe');
```

In SQLite, if we do not provide StudentID, it inserts NULL for that column. Normally, a primary key should reject that. But SQLite allows NULL in many primary-key columns unless you explicitly declare NOT NULL. SQLite's own documentation says this behavior is a legacy compatibility issue.

A primary key must have a value for every row and must be unique.

In a **real-world business setting**, DML statements are usually executed by applications behind the scenes. When an employee fills out a form and clicks **Save**, the application may execute an INSERT or UPDATE. When a record is removed, the application may execute a DELETE.

JDBC Student Database Application

Student Information

Student ID:101

Name:Alice Johnson

Major:Computer Science

Add Student

Update Student

Delete Student

Clear Fields

Student List

StudentID	Name	Major
101	Alice Johnson	Computer Science
102	Brian Smith	Biology
103	Carla Davis	Business
104	David Lee	Computer Science
105	Emma Wilson	Mathematics

Connected to MySQL database: SchoolDB

Student added successfully.

2. UPDATE

The **UPDATE operation** in SQL is used to **modify existing data in a table**.

For example, suppose we have this table with some values:

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Major VARCHAR(50)  
);  
INSERT INTO Student VALUES  
    (102, 'Alice Johnson', 'Computer Science'),  
    (103, 'Brian Smith', 'Biology'),  
    (104, 'Carla Davis', 'Business');
```

To change Brian's major:
UPDATE Student
 SET Major = 'Chemistry'
 WHERE StudentID = 103;

You can also update more than one column at the same time:

```
UPDATE Student  
  SET Name = 'Brian A. Smith',  
      Major = 'Physics'  
WHERE StudentID = 103;
```

You can also update multiple rows intentionally. For example:

```
UPDATE Student  
  SET Major = 'CSC'  
WHERE Major = 'Computer Science';
```

This would not work in MySQL due to Safe Update Mode enabled by default.

To disable it:

Go to **Edit → Preferences → SQL Editor**,
then find **Safe Updates (rejects UPDATES and DELETES with no restrictions)**.
Then reconnect to the SQL Server.

The **WHERE clause is very important** because it determines which rows will be changed.

For example:

```
UPDATE Student
```

```
    SET Major = 'Undeclared';
```

Because there is no WHERE clause, **every row in the table will be updated.**

Always check the WHERE clause carefully before executing an UPDATE statement.

Without a WHERE clause, the update applies to all rows in the table.

3. TRUNCATE AND DELETE

The TRUNCATE command removes all rows from a table but keeps the table itself.

For example, suppose we have this table with some values:

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Major VARCHAR(50)  
);  
INSERT INTO Student VALUES  
    (102, 'Alice Johnson', 'Computer Science'),  
    (103, 'Brian Smith', 'Biology'),  
    (104, 'Carla Davis', 'Business');
```

```
TRUNCATE TABLE Student;
```

Or

```
DELETE FROM Student;
```

This deletes all records from the Student table, but the table structure remains, including its columns and constraints.

SQLite does not support the TRUNCATE TABLE command.

Instead, use:

```
DELETE FROM Student;
```

TRUNCATE (or DELETE) and Foreign Keys

MySQL may prevent you from truncating a table if another table has a **foreign key that references it**.

In SQLite, by default, it does not enable foreign-key enforcement.

To enable foreign-key enforcement, you have to issue the PRAGMA command:

PRAGMA foreign_keys = ON;

*PRAGMA is a **SQLite-specific command** used to view or change certain SQLite settings and database behaviors. It is **not part of standard SQL**, so other DBMSs such as MySQL may not support PRAGMA.*

For example:

```
CREATE TABLE Department (  
  ID INT PRIMARY KEY,  
  Name VARCHAR(50)  
);
```

```
CREATE TABLE Student (  
  ID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  DeptID INT,  
  FOREIGN KEY (DeptID)  
    REFERENCES Department(ID)  
);
```

```
INSERT INTO Department VALUES  
(1, 'Computer Science'),  
(2, 'Cybersecurity'),  
(3, 'Math');
```

```
INSERT INTO Student VALUES  
(1, 'John Doe', 1),  
(2, 'Edgardo Guzman', 1),  
(3, 'Bill George', 1);
```

```
DELETE FROM Department;
```

*The script will fail in MySQL at the `TRUNCATE TABLE Department;` statement because `Department` is the **parent table** in a foreign key relationship and `Student` references it.*

In MySQL, `TRUNCATE TABLE Department` will fail because `Department` is referenced by a foreign key in the `Student` table. MySQL protects the referential relationship between the parent and child tables.

FOREIGN KEY constraint will be discussed more in the SQL-DQL lesson.

Using WHERE with DELETE

The WHERE clause is used with the DELETE command to specify **which rows should be removed** from a table.

Using the previous tables, we can issue a specific delete of a row that matches our filter (WHERE).

```
DELETE FROM Student  
WHERE ID = 2;
```

You can also use other columns in a table to filter which rows to delete, e.g.:

```
DELETE FROM Student  
WHERE Name = 'Edgardo Guzman';
```

If you intend to delete **one specific row**, using the **primary key** in the WHERE clause is usually the safest and clearest approach because the primary key uniquely identifies that row.

Using non-primary key columns is riskier because there could be more than one match with that value, so multiple rows could be deleted.

DDL vs. DML

A simple way to distinguish them is:

DDL → Defines the database and table structure

DML → Works with the data inside that structure

For example:

```
CREATE DATABASE StudentDB;      -- DDL
```

```
CREATE TABLE Student (...);   -- DDL
```

```
INSERT INTO Student ...;       -- DML
```

```
UPDATE Student ...;            -- DML
```

```
DELETE FROM Student ...;       -- DML
```

```
SELECT * FROM Student;         -- DQL
```